МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский ядерный университет «МИФИ»
(НИЯУ МИФИ)

ИНСТИТУТ ИНТЕЛЛЕКТУАЛЬНЫХ КИБЕРНЕТИЧЕСКИХ СИСТЕМ

КАФЕДРА «КОМПЬЮТЕРНЫЕ СИСТЕМЫ И ТЕХНОЛОГИИ» (№12)

ОТЧЕТ
КУРСОВОЙ ПРОЕКТ (СХЕМОТЕХНИКА ЦИФРОВЫХ УСТРОЙСТВ)

Семестр 5

Тема: «Сортировщик продукции на конвейере»

Студенты Коротаев С. А., Моря К. О. Группа С24-501
ФИО

Руководитель Решетько В. М.
ФИО, должность, звание, степень

Студент _____ Коротаев С. А.
подпись ФИО

Студент _____ Моря К. О.
подпись ФИО

Руководитель _____ Решетько В. М.
подпись ФИО

Оценка _____

Баллы, ECTS _____

«____»_____2026 г.

Москва 2026 г.

ОГЛАВЛЕНИЕ

1. Введение 3
2. Основные функции 3
3. Условное графическое обозначение и список портов ввода-вывода 3
4. Внутренняя структура устройства 4
4.1. Ядро сортировщика 4
4.2. Обёртка внешних датчиков 5
4.3. Демонстрационный генератор 5
4.4. Верхний уровень платы 5
4.5. Тестбенч 6
5. Принцип работы 6
6. Тактовая организация 8
7. Тестирование 8
8. Компиляция и синтез схемы 10
9. Результаты прошивки ПЛИС 12
10. Ограничения и допущения 12
11. Заключение 13
12. Приложение 14

1. Введение

Сортировщик определяет пригодность изделия по восьмиразрядному коду веса. Два датчика отмечают прохождение зоны измерения. Автомат фиксирует отсчёт и формирует импульс отбраковки, если вес выходит за допустимый диапазон.

2. Основные функции

1. Обнаружение изделия по любому из двух датчиков.
2. Фиксация веса после прохождения центра.
3. Проверка диапазона 95–105 включительно.
4. Отбраковка недопустимого изделия импульсом заданной длительности.
5. Ожидание освобождения датчиков перед следующим изделием.
6. Возврат в исходное состояние по активному низкому сбросу.

Разрешение конвейера постоянно равно единице, включая сброс и отбраковку.

3. Условное графическое обозначение и список портов ввода-вывода

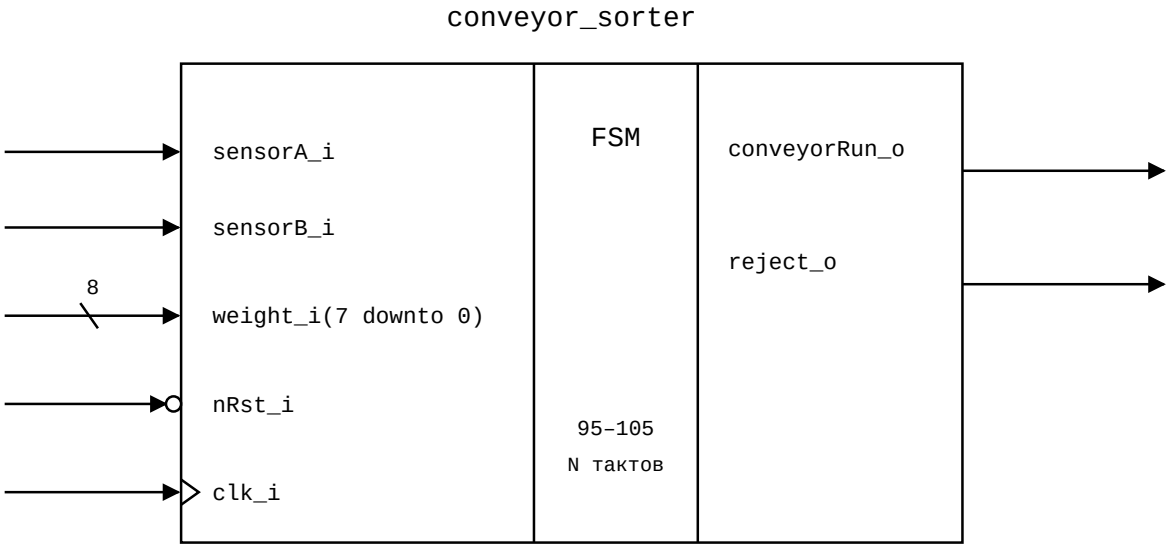


Рис. 1. УГО модуля conveyor_sorter

Таблица 1. Порты ядра

Сигнал	Направление	Разрядность	Назначение
clk_i	in	1	Положительный фронт такта
nRst_i	in	1	Асинхронный сброс при нуле
sensorA_i	in	1	Первый датчик; активен при 1
sensorB_i	in	1	Второй датчик; активен при 1
weight_i	in	8	Беззнаковый код (7 downto 0)
conveyorRun_o	out	1	Постоянное разрешение конвейера
reject_o	out	1	Импульс отбраковки

Треугольник обозначает тактовый вход, работающий по положительному фронту; кружок — активный низкий сброс; косая черта с числом 8 показывает ширину шины веса.

4. Внутренняя структура устройства

Ядро, генератор воздействий и верхний модуль используют общий такт. Состояние автомата и данные обновляются на положительном фронте.

4.1. Ядро сортировщика

conveyor_sorter.vhd содержит конечный автомат с пятью состояниями. Переходы FSM заданы оператором case в отдельном тактовом процессе. Регистры данных обновляются в одном общем тактовом процессе; для каждого регистра в нём предусмотрена собственная цепочка условий if/elsif. Все процессы синтезируемого кода тактовые, с асинхронным сбросом; process(all) и промежуточные сигналы следующего значения отсутствуют. Выходы ядра заданы конкурентными присваиваниями. Вес сравнивается непосредственно через IEEE.STD_LOGIC_UNSIGNED; NUMERIC_STD не подключается. Допустимый диапазон веса — 95–105 включительно; REJECT_PULSE_CYCLES = 5_000_000.

Таблица 2. Регистры ядра

Сигнал	Тип	Назначение
state_r	state_type	Текущее состояние
previousWeight_r	8 бит	Предыдущий отсчёт
measuredWeight_r	8 бит	Зафиксированный вес
centerFlag_r	1 бит	Признак прохождения центра
rejectCount_r	natural	Обратный счёт N − 1 … 0

4.2. Обёртка внешних датчиков

`conveyor_sorter_top.vhd` синхронизирует датчики двумя регистрами. Сброс устанавливается асинхронно и снимается через `resetSync_r`. Вес проходит через `weightMeta_r` и `weightSync_r`; `weightStable_r` обновляется при совпадении двух последовательных синхронизированных отсчётов. Совпадение отсчётов позволяет отсеять кратковременные изменения. При подключении внешних весов согласованность передачи всех восьми разрядов нужно обеспечивать отдельно. В демонстрационной сборке данная обёртка не используется.

4.3. Демонстрационный генератор

`flow_tester.vhd` уменьшает счётчик шага от STEP_CYCLES−1 до нуля и позиционный счётчик от 15 до нуля. Логическая позиция профиля равна 15−position_r и меняется раз в STEP_CYCLES = 5_000_000 тактов. При 50 МГц шаг длится 100 мс, одно изделие — 1,6 с. Флаг `rejectProduct_r` меняется при завершении профиля (position_r: 0 → 15): допустимый профиль с весом 100 чередуется с недопустимым весом 80. После сброса флаг равен нулю, первым идёт допустимое изделие. Выходы зарегистрированы и обновляются по фронту при входе в очередную позицию. Предварительного обнуления перед цепочкой условий нет: нулевой вес явно задаётся в ветви для пустых позиций и при сбросе.

Таблица 3. Активные позиции двух профилей

Позиция	A	B	Вес: допуст.	Вес: брак	Этап
2	1	0	20	20	Вход
3	1	0	50	30	Рост
4	1	1	80	60	Центр
5	1	1	100	80	Верхний отсчёт
6	1	1	98	78	Фиксация 100 / 80
7	0	1	70	50	Выход
8	0	1	30	30	Завершение

4.4. Верхний уровень платы

`qmtech_demo_top.vhd` соединяет генератор и ядро. Такт подаётся обоим модулям. Общий двухступенчатый синхронизатор обеспечивает асинхронное включение и синхронное снятие кнопочного сброса. Датчики и вес формируются внутри ПЛИС. Светодиодные выходы инвертированы: conveyorLed_n_o = not conveyorRun_s, rejectLed_n_o = not reject_s. Поэтому команда 1 в ядре даёт 0 на активном низком выходе индикации.

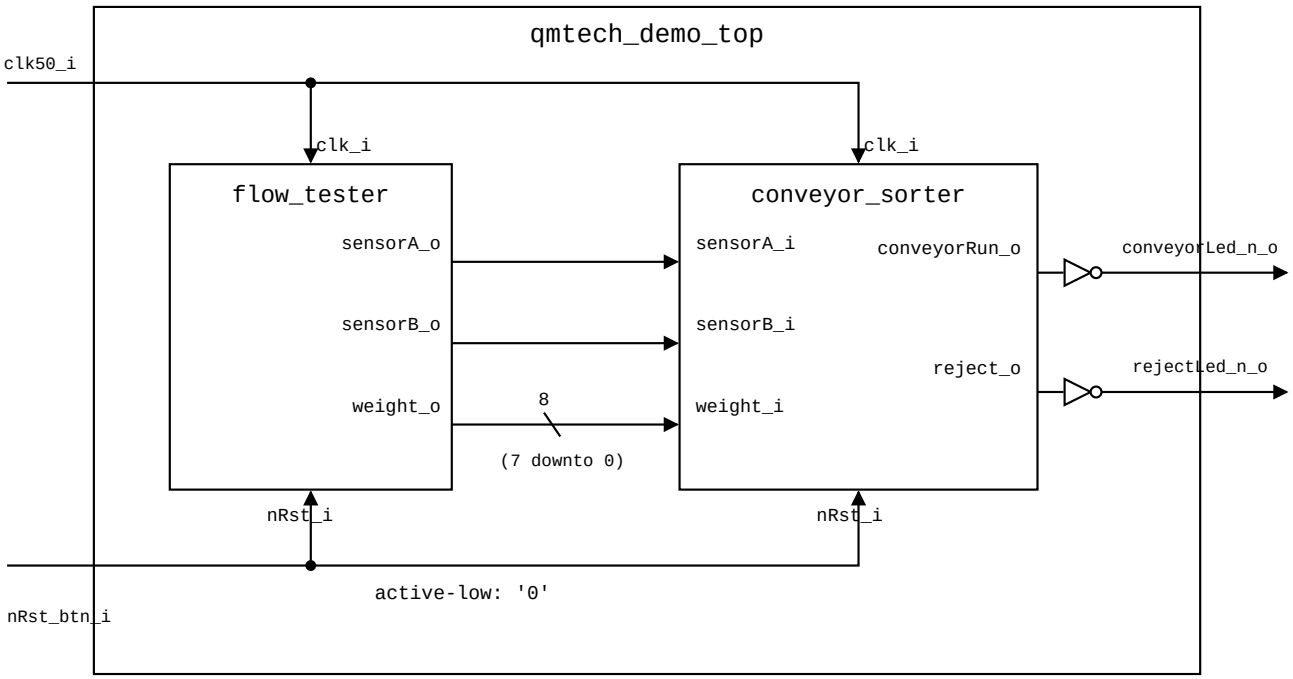


Рис. 2. Структура демонстрационного верхнего уровня

4.5. Тестбенч

`struct_tb.vhd` проверяет пять состояний автомата, границы весового диапазона, импульс отбраковки, ожидание освобождения датчиков, незавершённое измерение без обнаружения центра и асинхронный сброс во время отбраковки. Длительность импульса проверяется при REJECT_PULSE_CYCLES = 1, 2, 3 и 8; дополнительно проверены веса 0, 1 и 255 и завершение измерения при освобождении датчиков без спада веса. `flow_tester_tb.vhd` проверяет последовательность генератора совместно с ядром; `test_alternating.do` запускает семь вариантов обоих сценариев до самостоятельного завершения. Скрипт scripts/test.sh запускает те же проверки в GHDL и дополнительно проверяет отказ при неверном диапазоне веса.

5. Принцип работы

При nRst_i = '0' устанавливается WAIT_STATE; регистры веса, флаг центра и счётчик обнуляются. В WAIT_STATE `previousWeight_r` получает текущий вес. Любой активный датчик переводит автомат в MEASURE_STATE.

В MEASURE_STATE совместная активность датчиков устанавливает `centerFlag_r`. При первом спаде веса после отмеченного центра либо при спаде во время совместной активности датчиков фиксируется `previousWeight_r`. Если оба датчика

освободились после центра без спада, фиксируется тот же предыдущий отсчёт. До фиксации `previousWeight_r` обновляется на каждом фронте. `measuredWeight_r` получает отсчёт с предыдущего такта. Поэтому фиксируется значение перед первым спадом после центра, которое может отличаться от максимума за всё время прохождения изделия.

Обозначения: A и B — датчики; W — текущий вес; V — previousWeight_r; M — measuredWeight_r (весовые слова без знака); F — centerFlag_r; C — rejectCount_r; N — REJECT_PULSE_CYCLES.

P: W < V и (F = '1' или (A = '1' и B = '1')).
Q: A = '0' и B = '0' и F = '1'.

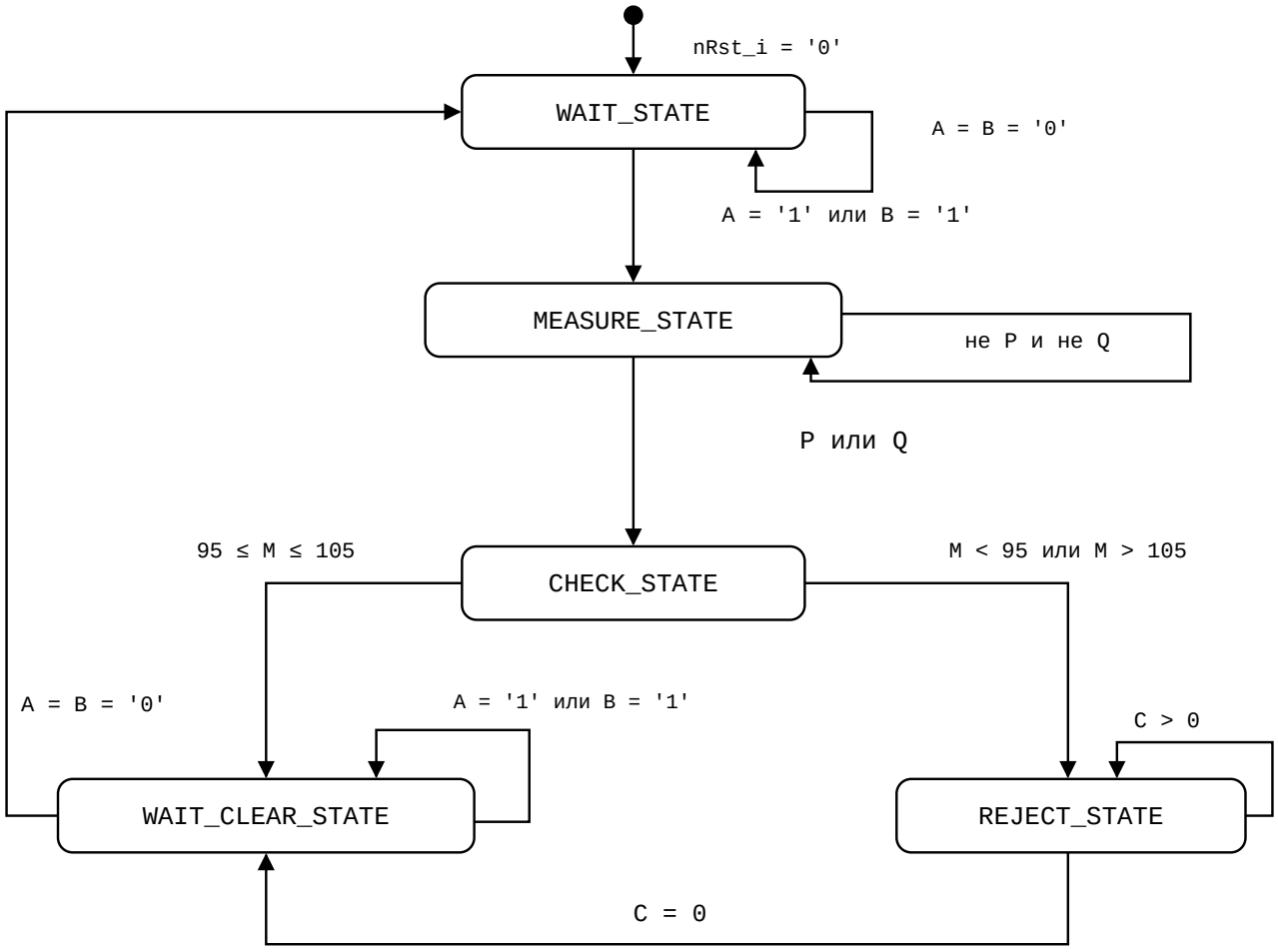


Рис. 3. Диаграмма состояний; сброс действует из любого состояния

Таблица 4. Условия переходов

Состояние	**Действия и переход**
`WAIT_STATE`	A = '1' или B = '1' → MEASURE_STATE; иначе ожидание. F ← 0, V ← W.
`MEASURE_STATE`	При P, иначе при Q: M ← V → CHECK_STATE. Иначе V ← W, состояние сохраняется. При A = B = '1' устанавливается F.
`CHECK_STATE`	95 ≤ M ≤ 105 → WAIT_CLEAR_STATE; иначе → REJECT_STATE; при проверке C ← N − 1.
`REJECT_STATE`	C = 0 → WAIT_CLEAR_STATE; иначе C ← C − 1, состояние сохраняется.
`WAIT_CLEAR_STATE`	A = B = '0' → WAIT_STATE; иначе ожидание. Данные сохраняются.

CHECK_STATE занимает один такт. reject_o = '1' только в REJECT_STATE, которое длится N тактов. После проверки и отбраковки автомат ждёт освобождения обоих датчиков. Асинхронный сброс имеет приоритет над тактом.

6. Тактовая организация

Номинальная частота такта — 50 МГц, период — 20 нс. Ядро и генератор работают от одного тактового сигнала; шаг генератора задаётся счётчиком без отдельного делённого такта.

Импульс отбраковки: 5_000_000 × 20 нс = 100 мс. В тесте: 3 × 20 нс = 60 нс. Параметр N положителен. Сброс ядра и генератора активный низкий, асинхронный. В демонстрации и обёртке внешних датчиков снятие кнопочного сброса синхронизировано двумя регистрами. В SDC заданы такт 20 нс, расчёт неопределённости и исключения для асинхронной кнопки и светодиодов без внешнего такта приёмника. Результаты временного анализа приведены далее.

7. Тестирование

Проверки `assert` с уровнем `severity failure` выполняются в семи самопроверяющихся запусках GHDL 6.0.0. Все семь запусков от 27.10.2026 завершились успешно; неверная конфигурация диапазона была отклонена ожидаемым сообщением. Для обнаружения зависаний предусмотрены тайм-ауты. Сценарии продуктов покрывают нижнюю и верхнюю границы допустимого веса и значения по обе стороны диапазона. Также проверяются длительность импульса, ожидание освобождения датчиков, измерение без фиксации центра и асинхронный сброс при активной отбраковке.

Таблица 5. Проверяемые сценарии ядра

Сценарий	**Входные условия**	**Ожидаемый результат**
Вес ниже диапазона	0, 1, 80 и 94	Отбраковка; 1/2/3/8 тактов
Нижняя граница	95	Изделие проходит без отбраковки
Верхняя граница	105	Изделие проходит без отбраковки
Вес выше диапазона	106 и 255	Отбраковка; 1/2/3/8 тактов
Нет центра	Датчик A срабатывает без совместного обнаружения	Измерение не завершается
Сброс во время импульса	Асинхронный сброс при reject = 1	reject немедленно сбрасывается

Для совместной проверки генератора и сортировщика использован `flow_tester_tb.vhd`. Проверены STEP_CYCLES = 1, 2 и 8, REJECT_PULSE_CYCLES = 3, период такта — 20 нс. Датчики и все значения профиля проверяются на каждом такте. На рисунке представлен запуск с STEP_CYCLES = 8. Один профиль изделия занимает 16 × 8 × 20 нс = 2,56 мкс; расчётная длительность импульса отбраковки — 60 нс. Ускорение позволяет просмотреть несколько изделий за один запуск.

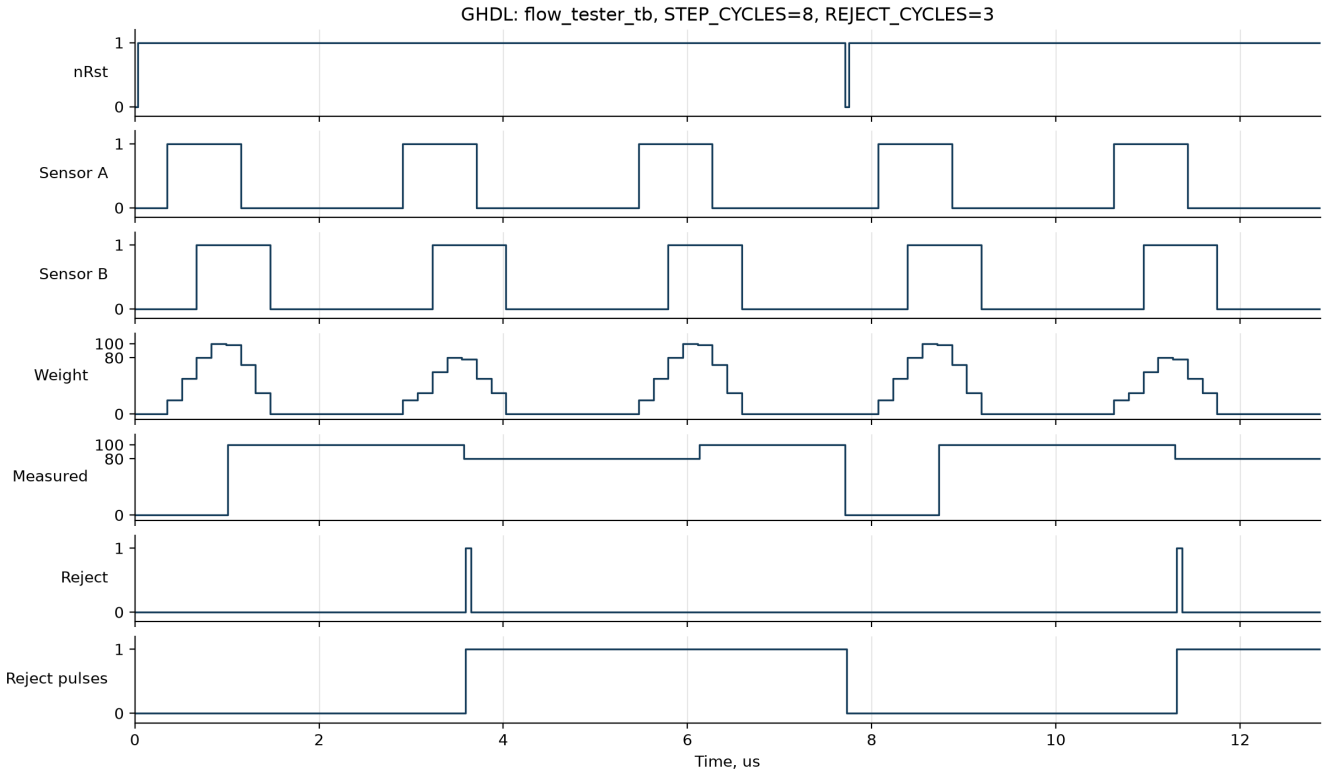


Рис. 4. Временная диаграмма совместной работы генератора и сортировщика, flow_tester_tb

На рисунке видны пять проходов изделия. В первых трёх зафиксированный вес равен 100, 80 и 100. При весе 100 сигнал отбраковки остаётся равным нулю; при весе 80 появляется импульс, а счётчик импульсов принимает значение 1. Между третьим и четвёртым проходами подан активный низкий сброс. Счётчик обнуляется, и

последовательность начинается заново: сначала вес 100 без отбраковки, затем вес 80 с импульсом. Разрешение конвейера на всём показанном интервале равно единице.

Диаграмма построена из VCD текущего запуска GHDL. Масштаб общего графика не предназначен для измерения импульса 60 нс; точная длительность подтверждена потактовыми проверками. Протоколы сохранены в docs/verification/ghdl.log. Questa 2025.2 скомпилировала все исходники без ошибок и предупреждений; запуск Questa в данной среде ограничен отсутствием лицензии.

8. Компиляция и синтез схемы

Актуальный проект полностью собран 27.10.2026 в Quartus Prime Standard 25.1std.0 Build 1129 для Cyclone 10 LP 10CL080YU484C8G. Полная компиляция завершилась без ошибок и критических предупреждений. Получен файл qmtech_demo.sof. Итоговые показатели и протоколы сохранены в docs/verification/.
Результаты актуальной сборки

Показатель	**Значение**
Логические элементы после размещения	134 / 81 264
Регистры	79
Период / частота такта	20 нс / 50 МГц
Fmax, Slow 1200 mV 85 °C	187.3 МГц
Минимальный setup slack	14.661 нс
Минимальный hold slack	0.183 нс
Неограниченные входные / выходные пути	0 / 0

Постоянный нулевой выход D1 вызывает ожидаемое предупреждение об оптимизации константы. Также Quartus напоминает о требованиях к электрическому сопряжению 3,3 В; на штатной плате светодиоды подключены через резисторы 1 кОм. В QSF явно задан драйвер 4 мА. Эти сообщения не являются ошибками логики или временного анализа.

Следующий исторический снимок Flow Summary относится к прежней версии (119 LE, 69 регистров) и не используется как подтверждение актуальных показателей.

Compilation Report - qmtech_demo

Flow Summary

<<Filter>>

Flow Status	Successful - Wed Oct 07 10:43:12 2026
Quartus Prime Version	20.1.1 Build 720 11/11/2020 SJ Lite Edition
Revision Name	qmtech_demo
Top-level Entity Name	qmtech_demo_top
Family	Cyclone 10 LP
Device	10CL080YU484C8G
Timing Models	Final
Total logic elements	119 / 81,264 (< 1 %)
Total registers	69
Total pins	4 / 290 (1 %)
Total virtual pins	0
Total memory bits	0 / 2,810,880 (0 %)
Embedded Multiplier 9-bit elements	0 / 488 (0 %)
Total PLLs	0 / 4 (0 %)

Рис. 5. Историческая сводка предыдущей версии

Приведённые далее снимки RTL и технологических схем также относятся к предыдущей версии проекта. Они служат иллюстрацией структуры и не подтверждают результаты сборки текущих исходников.

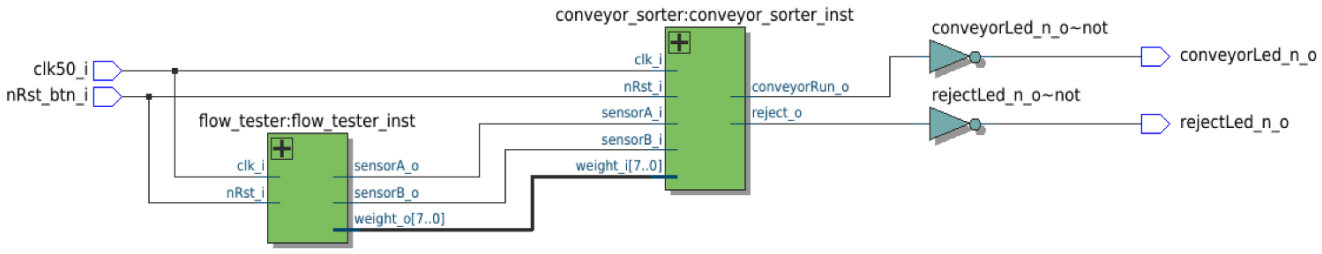


Рис. 6. RTL-схема: генератор, ядро и инверторы выходов

В RTL-схеме датчики и шина веса соединяют генератор с ядром. В технологической схеме постоянный выход conveyorLed_n_o оптимизирован до нуля, а rejectLed_n_o формируется инверсией признака REJECT_STATE.

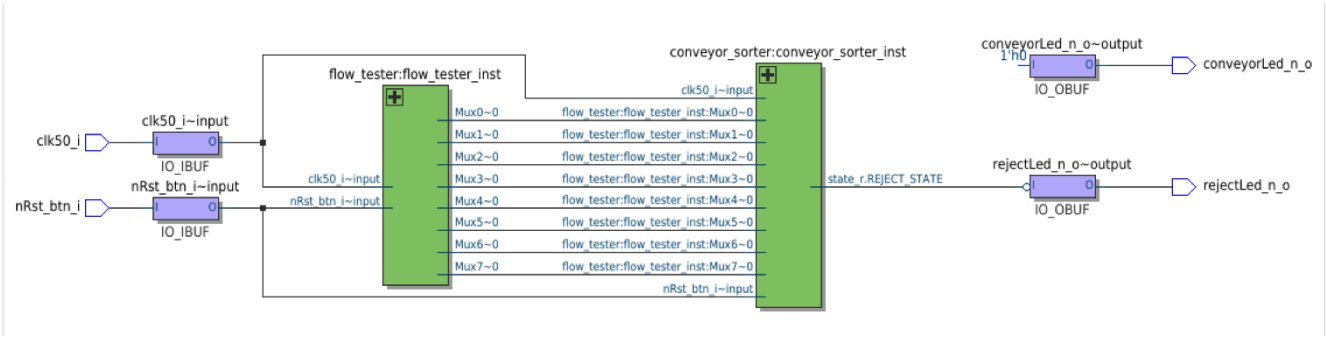


Рис. 7. Технологическая схема после синтеза (Post-Mapping)

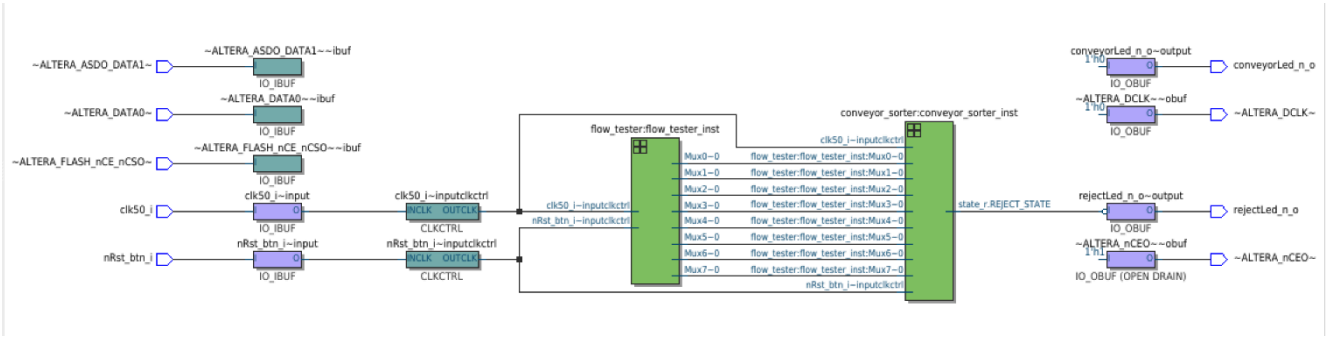


Рис. 8. Технологическая схема после размещения (Post-Fitting)

9. Результаты прошивки ПЛИС

В файле QSF указаны контакты clk50_i — PIN_G1, nRst_btn_i — PIN_P4, conveyorLed_n_o — PIN_V10, rejectLed_n_o — PIN_U9 и стандарт 3.3-V LVTTL. Приложенный снимок Pin Planner относится к предыдущей версии проекта. Назначения G1/P4/V10/U9 и активный низкий уровень светодиодов сверены с опубликованной схемой QMTECH V03 в репозитории производителя ChinaQMTECH/CYCLONE_10_STARTER_KIT. Актуальная сборка учитывает эти назначения. Перед загрузкой необходимо проверить ревизию фактически используемой платы.

При такте 50 МГц ожидается постоянный сигнал конвейера и импульс отбраковки 100 мс один раз в 3,2 с. Результаты загрузки конфигурации и испытания платы не предоставлены.

10. Ограничения и допущения

Вес — код 0 … 255 без заданных физических единиц. Изделия проходят последовательно, освобождая оба датчика. Без совместного срабатывания центр не фиксируется и измерение может не завершиться. Тайм-аут, фильтрация шумов и аварийная остановка отсутствуют. Первый спад после центра завершает

измерение, даже если вызван шумом. Силовая часть отбраковки не описана. Демонстрационный генератор чередует два заданных профиля; реальные датчики и весы не подключены.

11. Заключение

Сортировщик содержит автомат с пятью состояниями, проверяет диапазон 95–105 и формирует импульс отбраковки ровно N тактов. По замечаниям преподавателя применён STD_LOGIC_UNSIGNED, счётчики переведены на обратный счёт до нуля, процессы обновления данных и выходов генератора выполняются по такту без process(all) и предварительных заглушек. Семь симуляций GHDL прошли успешно, включая граничные длительности и веса; неверный диапазон отклоняется. Quartus выполнил полную компиляцию и временной анализ для 50 МГц без ошибок и критических предупреждений. Исторические снимки помечены отдельно. Физическая прошивка и испытание платы остаются обязательным завершающим этапом.

12. Приложение

Полные VHDL-исходники, сценарий проверки и настройки проекта. Номера слева соответствуют исходным строкам; длинные строки перенесены при отображении.

Приложение 1. Файл conveyor_sorter.vhd

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  entity conveyor_sorter is
6      generic (
7          WEIGHT_MIN          : natural range 0 to 255 := 95;
8          WEIGHT_MAX          : natural range 0 to 255 := 105;
9          REJECT_PULSE_CYCLES : positive := 5_000_000
10     );
11     port (
12         clk_i         : in  std_logic;
13         nRst_i        : in  std_logic;
14         sensorA_i     : in  std_logic;
15         sensorB_i     : in  std_logic;
16         weight_i      : in  std_logic_vector(7 downto 0);
17         conveyorRun_o : out std_logic;
18         reject_o      : out std_logic
19     );
20 end conveyor_sorter;
21
22 architecture rtl of conveyor_sorter is
23
24     type state_type is (WAIT_STATE, MEASURE_STATE, CHECK_STATE,
   REJECT_STATE, WAIT_CLEAR_STATE);
25
26     signal state_r          : state_type;
27     signal previousWeight_r : std_logic_vector(7 downto 0);
28     signal measuredWeight_r : std_logic_vector(7 downto 0);
29     signal rejectCount_r    : natural range 0 to REJECT_PULSE_CYCLES - 1;
30     signal centerFlag_r     : std_logic;
31
32 begin
33
34     assert WEIGHT_MIN <= WEIGHT_MAX
35         report "WEIGHT_MIN must not exceed WEIGHT_MAX" severity failure;
36
37     state_transitions : process(clk_i, nRst_i)
38     begin
39         if nRst_i = '0' then
40             state_r <= WAIT_STATE;
41         elsif rising_edge(clk_i) then
42             case state_r is
43                 when WAIT_STATE =>
44                     if sensorA_i = '1' or sensorB_i = '1' then
45                         state_r <= MEASURE_STATE;
46                     end if;
47                 when MEASURE_STATE =>
48                     if weight_i < previousWeight_r
49                        and (centerFlag_r = '1' or (sensorA_i = '1' and
   sensorB_i = '1')) then
50                         state_r <= CHECK_STATE;
51                     elsif sensorA_i = '0' and sensorB_i = '0' and
   centerFlag_r = '1' then
52                         state_r <= CHECK_STATE;
53                     end if;
54                 when CHECK_STATE =>
55                     if measuredWeight_r >= WEIGHT_MIN and measuredWeight_r
   <= WEIGHT_MAX then
56                         state_r <= WAIT_CLEAR_STATE;
57                     else
58                         state_r <= REJECT_STATE;
59                     end if;
60                 when REJECT_STATE =>
61                     if rejectCount_r = 0 then
62                         state_r <= WAIT_CLEAR_STATE;
63                     end if;
64                 when WAIT_CLEAR_STATE =>
```

```
                    if sensorA_i = '0' and sensorB_i = '0' then
                        state_r <= WAIT_STATE;
                    end if;
            end case;
        end if;
    end process;

    data_registers : process(clk_i, nRst_i)
    begin
        if nRst_i = '0' then
            previousWeight_r <= (others => '0');
            measuredWeight_r <= (others => '0');
            rejectCount_r    <= 0;
            centerFlag_r     <= '0';
        elsif rising_edge(clk_i) then
            if state_r = WAIT_STATE then
                previousWeight_r <= weight_i;
            elsif state_r = MEASURE_STATE
               and not ((weight_i < previousWeight_r
                         and (centerFlag_r = '1' or (sensorA_i = '1' and sensorB_i = '1')))
                        or (sensorA_i = '0' and sensorB_i = '0' and centerFlag_r = '1')) then
                previousWeight_r <= weight_i;
            end if;

            if state_r = MEASURE_STATE
               and ((weight_i < previousWeight_r
                     and (centerFlag_r = '1' or (sensorA_i = '1' and sensorB_i = '1')))
                    or (sensorA_i = '0' and sensorB_i = '0' and centerFlag_r = '1')) then
                measuredWeight_r <= previousWeight_r;
            end if;

            -- Entry to REJECT_STATE starts the first of N active cycles.
            -- Loading N-1 also handles N=1 without underflow or an extra cycle.
            if state_r = CHECK_STATE then
                rejectCount_r <= REJECT_PULSE_CYCLES - 1;
            elsif state_r = REJECT_STATE and rejectCount_r /= 0 then
                rejectCount_r <= rejectCount_r - 1;
            end if;

            if state_r = WAIT_STATE then
                centerFlag_r <= '0';
            elsif state_r = MEASURE_STATE and sensorA_i = '1' and sensorB_i = '1' then
                centerFlag_r <= '1';
            end if;
        end if;
    end process;

    conveyorRun_o <= '1';
    reject_o <= '1' when state_r = REJECT_STATE else '0';

end rtl;
```

Приложение 2. Файл conveyor_sorter_top.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity conveyor_sorter_top is
    port (
        clk50_i       : in  std_logic;
        nRst_btn_i    : in  std_logic;
        sensorA_i     : in  std_logic;
        sensorB_i     : in  std_logic;
        weight_i      : in  std_logic_vector(7 downto 0);
        conveyorRun_o : out std_logic;
        reject_o      : out std_logic;
        conveyorLed_n_o : out std_logic;
        rejectLed_n_o   : out std_logic
    );
end conveyor_sorter_top;

architecture Structural of conveyor_sorter_top is

    signal resetSync_r      : std_logic_vector(1 downto 0);
    signal sensorASync_r    : std_logic_vector(1 downto 0);
    signal sensorBSync_r    : std_logic_vector(1 downto 0);

    signal weightMeta_r     : std_logic_vector(7 downto 0);
    signal weightSync_r     : std_logic_vector(7 downto 0);
    signal weightPrevious_r : std_logic_vector(7 downto 0);
    signal weightStable_r   : std_logic_vector(7 downto 0);

    signal nRst_s           : std_logic;
    signal conveyorRun_s    : std_logic;
    signal reject_s         : std_logic;

begin

    process(clk50_i, nRst_btn_i)
    begin

        if nRst_btn_i = '0' then

            resetSync_r <= (others => '0');

        elsif rising_edge(clk50_i) then

            resetSync_r(0) <= '1';
            resetSync_r(1) <= resetSync_r(0);

        end if;

    end process;

    nRst_s <= resetSync_r(1);

    process(clk50_i, nRst_s)
    begin

        if nRst_s = '0' then

            sensorASync_r    <= (others => '0');
            sensorBSync_r    <= (others => '0');
            weightMeta_r     <= (others => '0');
            weightSync_r     <= (others => '0');
            weightPrevious_r <= (others => '0');
            weightStable_r   <= (others => '0');

        elsif rising_edge(clk50_i) then

            sensorASync_r(0) <= sensorA_i;
            sensorASync_r(1) <= sensorASync_r(0);

            sensorBSync_r(0) <= sensorB_i;
            sensorBSync_r(1) <= sensorBSync_r(0);

            weightMeta_r <= weight_i;
            weightSync_r <= weightMeta_r;

            if weightSync_r = weightPrevious_r then
                weightStable_r <= weightSync_r;
            end if;
```

```
79
80              weightPrevious_r <= weightSync_r;
81
82          end if;
83
84      end process;
85
86      dut_inst : entity work.conveyor_sorter
87          generic map (
88              WEIGHT_MIN          => 95,
89              WEIGHT_MAX          => 105,
90              REJECT_PULSE_CYCLES => 5_000_000
91          )
92          port map (
93              clk_i         => clk50_i,
94              nRst_i        => nRst_s,
95              sensorA_i     => sensorASync_r(1),
96              sensorB_i     => sensorBSync_r(1),
97              weight_i      => weightStable_r,
98              conveyorRun_o => conveyorRun_s,
99              reject_o      => reject_s
100         );
101
102     conveyorRun_o <= conveyorRun_s;
103     reject_o      <= reject_s;
104
105     conveyorLed_n_o <= not conveyorRun_s;
106     rejectLed_n_o   <= not reject_s;
107
108 end Structural;
```

Приложение 3. Файл flow_tester.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity flow_tester is
    generic (STEP_CYCLES : positive := 5_000_000);
    port (
        clk_i     : in  std_logic;
        nRst_i    : in  std_logic;
        sensorA_o : out std_logic;
        sensorB_o : out std_logic;
        weight_o  : out std_logic_vector(7 downto 0)
    );
end flow_tester;

architecture rtl of flow_tester is
    signal stepCount_r : natural range 0 to STEP_CYCLES - 1 := STEP_CYCLES - 1;
    -- Countdown 15..0 corresponds to logical profile positions 0..15.
    signal position_r : natural range 0 to 15 := 15;
    signal rejectProduct_r : std_logic;
begin
    counter_registers : process(clk_i, nRst_i)
    begin
        if nRst_i = '0' then
            stepCount_r <= STEP_CYCLES - 1;
            position_r <= 15;
            rejectProduct_r <= '0';
        elsif rising_edge(clk_i) then
            if stepCount_r = 0 then
                stepCount_r <= STEP_CYCLES - 1;
            else
                stepCount_r <= stepCount_r - 1;
            end if;

            if stepCount_r = 0 then
                if position_r = 0 then
                    position_r <= 15;
                    rejectProduct_r <= not rejectProduct_r;
                else
                    position_r <= position_r - 1;
                end if;
            end if;
        end if;
    end process;

    output_registers : process(clk_i, nRst_i)
    begin
        if nRst_i = '0' then
            sensorA_o <= '0';
            sensorB_o <= '0';
            weight_o <= x"00";
        elsif rising_edge(clk_i) then
            if stepCount_r = 0 then
                -- Decode the position being entered on this same edge.
                -- Old countdown 14 means the next logical position is 2.
                if position_r >= 10 and position_r <= 14 then
                    sensorA_o <= '1';
                else
                    sensorA_o <= '0';
                end if;

                if position_r >= 8 and position_r <= 12 then
                    sensorB_o <= '1';
                else
                    sensorB_o <= '0';
                end if;

                if position_r = 14 then
                    weight_o <= x"14"; -- 20
                elsif position_r = 13 and rejectProduct_r = '0' then
                    weight_o <= x"32"; -- 50
                elsif position_r = 13 then
                    weight_o <= x"1E"; -- 30
                elsif position_r = 12 and rejectProduct_r = '0' then
                    weight_o <= x"50"; -- 80
                elsif position_r = 12 then
                    weight_o <= x"3C"; -- 60
```

```
                elsif position_r = 11 and rejectProduct_r = '0' then
                    weight_o <= x"64"; -- 100
                elsif position_r = 11 then
                    weight_o <= x"50"; -- 80
                elsif position_r = 10 and rejectProduct_r = '0' then
                    weight_o <= x"62"; -- 98
                elsif position_r = 10 then
                    weight_o <= x"4E"; -- 78
                elsif position_r = 9 and rejectProduct_r = '0' then
                    weight_o <= x"46"; -- 70
                elsif position_r = 9 then
                    weight_o <= x"32"; -- 50
                elsif position_r = 8 then
                    weight_o <= x"1E"; -- 30
                else
                    -- Empty positions are a real zero-weight stimulus.
                    weight_o <= x"00";
                end if;
            end if;
        end if;
    end process;
end rtl;
```

Приложение 4. Файл qmtech_demo_top.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity qmtech_demo_top is
    port (clk50_i : in std_logic; nRst_btn_i : in std_logic; conveyorLed_n_o : out std_logic;
    rejectLed_n_o : out std_logic);
end qmtech_demo_top;

architecture structural of qmtech_demo_top is
    signal sensorA_s, sensorB_s, conveyorRun_s, reject_s : std_logic;
    signal resetSync_r : std_logic_vector(1 downto 0);
    signal nRst_s : std_logic;
    signal weight_s : std_logic_vector(7 downto 0);
begin
    -- Asynchronous assertion, synchronous release shared by both modules.
    reset_synchronizer : process(clk50_i, nRst_btn_i)
    begin
        if nRst_btn_i = '0' then
            resetSync_r <= (others => '0');
        elsif rising_edge(clk50_i) then
            resetSync_r(0) <= '1';
            resetSync_r(1) <= resetSync_r(0);
        end if;
    end process;
    nRst_s <= resetSync_r(1);

    flow_tester_inst : entity work.flow_tester
        generic map (STEP_CYCLES => 5_000_000)
        port map (clk_i => clk50_i, nRst_i => nRst_s, sensorA_o => sensorA_s, sensorB_o =>
    sensorB_s, weight_o => weight_s);
    conveyor_sorter_inst : entity work.conveyor_sorter
        generic map (REJECT_PULSE_CYCLES => 5_000_000)
        port map (clk_i => clk50_i, nRst_i => nRst_s, sensorA_i => sensorA_s, sensorB_i =>
    sensorB_s, weight_i => weight_s, conveyorRun_o => conveyorRun_s, reject_o => reject_s);
    conveyorLed_n_o <= not conveyorRun_s;
    rejectLed_n_o <= not reject_s;
end structural;
```

Приложение 5. Файл struct_tb.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity struct_tb is
    generic (REJECT_CYCLES : positive := 3);
end struct_tb;

architecture simulation of struct_tb is
    constant CLOCK_PERIOD : time := 20 ns;
    signal done_s        : boolean := false;
    signal clk_s         : std_logic := '0';
    signal nRst_s        : std_logic := '0';
    signal sensorA_s     : std_logic := '0';
    signal sensorB_s     : std_logic := '0';
    signal weight_s      : std_logic_vector(7 downto 0) := (others => '0');
    signal conveyorRun_s : std_logic;
    signal reject_s      : std_logic;
begin
    dut_inst : entity work.conveyor_sorter
        generic map (REJECT_PULSE_CYCLES => REJECT_CYCLES)
        port map (clk_i => clk_s, nRst_i => nRst_s, sensorA_i => sensorA_s, sensorB_i =>
    sensorB_s,
                  weight_i => weight_s, conveyorRun_o => conveyorRun_s, reject_o => reject_s);

    clock_generator : process
    begin
        while not done_s loop
            clk_s <= '0'; wait for CLOCK_PERIOD / 2;
            clk_s <= '1'; wait for CLOCK_PERIOD / 2;
        end loop;
        wait;
    end process;

    watchdog : process
    begin
        wait until done_s for 100 us;
        assert done_s report "struct_tb timeout" severity failure;
        wait;
    end process;

    stimulus_and_check : process
        procedure tick is
        begin
            wait until rising_edge(clk_s);
            wait for 1 ns;
        end procedure;

        procedure reset_dut is
        begin
            nRst_s <= '0';
            wait for 1 ns;
            assert conveyorRun_s = '1' and reject_s = '0'
                report "Reset must keep the conveyor enabled and reject inactive" severity
    failure;
            tick;
            nRst_s <= '1';
            tick;
        end procedure;

        procedure check_product(target_weight : natural; expected_reject : std_logic) is
        begin
            weight_s <=
    IEEE.STD_LOGIC_ARITH.CONV_STD_LOGIC_VECTOR(target_weight,
    weight_s'length);
            sensorA_s <= '1';
            sensorB_s <= '0';
            tick;
            sensorB_s <= '1';
            tick;
            weight_s <=
    IEEE.STD_LOGIC_ARITH.CONV_STD_LOGIC_VECTOR(target_weight - 1,
    weight_s'length);
```

```
68              tick;
69              assert reject_s = '0'
70                  report "Reject must remain inactive during the check
    state" severity failure;
71              tick;
72              assert reject_s = expected_reject
73                  report "Product was classified incorrectly" severity
    failure;
74
75              if expected_reject = '1' then
76                  for cycle in 2 to REJECT_CYCLES loop
77                      tick;
78                      assert reject_s = '1'
79                          report "Reject pulse ended before the configured
    duration" severity
    failure;
80                  end loop;
81                  tick;
82                  assert reject_s = '0'
83                      report "Reject pulse exceeded the configured duration"
    severity failure;
84              else
85                  tick;
86                  assert reject_s = '0'
87                      report "Valid product entered the reject state"
    severity failure;
88              end if;
89
90              tick;
91              assert reject_s = '0'
92                  report "Reject must remain inactive while waiting for
    sensor release" severity
    failure;
93              sensorA_s <= '0';
94              sensorB_s <= '0';
95              tick;
96              tick;
97              assert conveyorRun_s = '1' and reject_s = '0'
98                  report "Sorter did not return to the waiting state"
    severity failure;
99          end procedure;
100     begin
101         reset_dut;
102         check_product(94, '1');
103         check_product(95, '0');
104         check_product(105, '0');
105         check_product(106, '1');
106         check_product(80, '1');
107         check_product(255, '1');
108         check_product(1, '1');
109
110         reset_dut;
111         weight_s <= x"00";
112         sensorA_s <= '1';
113         sensorB_s <= '1';
114         tick;
115         tick;
116         sensorA_s <= '0';
117         sensorB_s <= '0';
118         tick;
119         assert reject_s = '0' report "Zero weight must pass through
    CHECK_STATE" severity
    failure;
120         tick;
121         for cycle in 1 to REJECT_CYCLES loop
122             assert reject_s = '1' report "Zero weight must be rejected for
    exactly N cycles"
    severity failure;
123             tick;
124         end loop;
125         assert reject_s = '0' report "Zero-weight rejection did not end"
    severity failure;
126         tick;
127         tick;
128         check_product(100, '0');
129
130         reset_dut;
131         weight_s <= x"6E";
132         sensorA_s <= '1';
133         tick;
```

```
        sensorA_s <= '0';
        weight_s <= (others => '0');
        tick;
        tick;
        assert reject_s = '0'
            report "Measurement without center detection must not be completed" severity failure;

        reset_dut;
        weight_s <= x"50";
        sensorA_s <= '1';
        tick;
        sensorB_s <= '1';
        tick;
        weight_s <= x"4F";
        tick;
        tick;
        assert reject_s = '1'
            report "Out-of-range product must enter the reject state" severity failure;
        nRst_s <= '0';
        wait for 1 ns;
        assert conveyorRun_s = '1' and reject_s = '0'
            report "Asynchronous reset must stop rejection immediately" severity failure;
        tick;
        nRst_s <= '1';
        sensorA_s <= '0';
        sensorB_s <= '0';
        tick;
        assert reject_s = '0'
            report "Sorter must remain reset after an interrupted reject pulse" severity failure;

        report "All conveyor sorter checks passed" severity note;
        done_s <= true;
        wait;
    end process;
end simulation;
```

Приложение 6. Файл flow_tester_tb.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity flow_tester_tb is
    generic (STEP_CYCLES : positive := 8);
end flow_tester_tb;

architecture simulation of flow_tester_tb is
    constant CLOCK_PERIOD : time := 20 ns;
    constant REJECT_CYCLES : positive := 3;
    signal done_s : boolean := false;
    signal clk_s : std_logic := '0';
    signal nRst_s : std_logic := '0';
    signal sensorA_s, sensorB_s, conveyorRun_s, reject_s : std_logic;
    signal weight_s : std_logic_vector(7 downto 0);
    signal rejectPulses_s : natural := 0;
begin
    clock_generator : process
    begin
        while not done_s loop
            clk_s <= '0'; wait for CLOCK_PERIOD / 2;
            clk_s <= '1'; wait for CLOCK_PERIOD / 2;
        end loop;
        wait;
    end process;

    generator_inst : entity work.flow_tester
        generic map (STEP_CYCLES => STEP_CYCLES)
        port map (clk_i => clk_s, nRst_i => nRst_s,
                  sensorA_o => sensorA_s, sensorB_o => sensorB_s, weight_o => weight_s);

    sorter_inst : entity work.conveyor_sorter
        generic map (REJECT_PULSE_CYCLES => REJECT_CYCLES)
        port map (clk_i => clk_s, nRst_i => nRst_s,
```

```
36                      sensorA_i => sensorA_s, sensorB_i => sensorB_s, weight_i
   => weight_s,
37                      conveyorRun_o => conveyorRun_s, reject_o => reject_s);
38
39      monitor : process
40          variable previousReject_v : std_logic := '0';
41          variable pulseCycles_v : natural := 0;
42      begin
43          wait until rising_edge(clk_s);
44          wait for 1 ns;
45          assert conveyorRun_s = '1' report "Conveyor must remain enabled"
   severity failure;
46          if nRst_s = '0' then
47              rejectPulses_s <= 0;
48              previousReject_v := '0';
49              pulseCycles_v := 0;
50              assert reject_s = '0' report "Reject must be inactive during
   reset" severity
      failure;
51          else
52              if reject_s = '1' then
53                  if previousReject_v = '0' then
54                      rejectPulses_s <= rejectPulses_s + 1;
55                  end if;
56                  pulseCycles_v := pulseCycles_v + 1;
57              elsif previousReject_v = '1' then
58                  assert pulseCycles_v = REJECT_CYCLES
59                      report "Reject pulse has incorrect duration" severity
   failure;
60                  pulseCycles_v := 0;
61              end if;
62              previousReject_v := reject_s;
63          end if;
64      end process;
65
66      watchdog : process
67      begin
68          wait until done_s for 1 ms;
69          assert done_s report "flow_tester_tb timeout" severity failure;
70          wait;
71      end process;
72
73      stimulus : process
74          procedure tick is
75          begin
76              wait until rising_edge(clk_s);
77              wait for 2 ns;
78          end procedure;
79
80          procedure check_product(productIndex : natural) is
81              variable expectedPeak_v : natural;
82              variable position_v : natural;
83              variable expectedWeight_v : natural;
84              type profile_type is array (0 to 15) of natural;
85              constant VALID_PROFILE : profile_type := (0, 0, 20, 50, 80,
   100, 98, 70, 30, 0, 0,
      0, 0, 0, 0, 0);
86              constant INVALID_PROFILE : profile_type := (0, 0, 20, 30, 60,
   80, 78, 50, 30, 0, 0,
      0, 0, 0, 0, 0);
87          begin
88              if productIndex mod 2 = 0 then
89                  expectedPeak_v := 100;
90              else
91                  expectedPeak_v := 80;
92              end if;
93              for cycle in 1 to 16 * STEP_CYCLES loop
94                  tick;
95                  position_v := (cycle / STEP_CYCLES) mod 16;
96                  if productIndex mod 2 = 0 then
97                      expectedWeight_v := VALID_PROFILE(position_v);
98                  else
99                      expectedWeight_v := INVALID_PROFILE(position_v);
100                 end if;
101                 assert conv_integer(weight_s) = expectedWeight_v
102                     report "Weight profile differs at a clock edge"
   severity failure;
103                 if position_v >= 2 and position_v <= 6 then
104                     assert sensorA_s = '1' report "Sensor A missing"
   severity failure;
```

```
                else
                    assert sensorA_s = '0' report "Unexpected sensor A"
severity failure;
                end if;
                if position_v >= 4 and position_v <= 8 then
                    assert sensorB_s = '1' report "Sensor B missing"
severity failure;
                else
                    assert sensorB_s = '0' report "Unexpected sensor B"
severity failure;
                end if;
                if cycle = 5 * STEP_CYCLES then
                    assert sensorA_s = '1' and sensorB_s = '1'
                        report "Both sensors must be active at the peak"
severity failure;
                    assert conv_integer(weight_s) = expectedPeak_v
                        report "Incorrect alternating weight profile"
severity failure;
                end if;
            end loop;
            assert rejectPulses_s = (productIndex + 1) / 2
                report "Only alternate, invalid products must be rejected"
severity failure;
            assert reject_s = '0' report "Reject must end before the next
product" severity
    failure;
        end procedure;
    begin
        tick;
        tick;
        nRst_s <= '1';
        -- Valid, invalid, valid: the next scheduled profile would be
invalid.
        for product in 0 to 2 loop
            check_product(product);
        end loop;
        -- Reset must restore the valid-first sequence.
        nRst_s <= '0';
        tick;
        tick;
        nRst_s <= '1';
        check_product(0);
        check_product(1);
        report "All alternating flow checks passed" severity note;
        done_s <= true;
        wait;
    end process;
end simulation;
```

Приложение 7. Файл test_alternating.do

```
onerror {quit -code 1 -force}
onbreak {quit -code 1 -force}
set source_dir [file dirname [file normalize [info script]]]
file mkdir [file join $source_dir simulation alternating]
cd [file join $source_dir simulation alternating]
if {![file exists work]} {vlib work}
vmap work work
foreach source {conveyor_sorter.vhd flow_tester.vhd
conveyor_sorter_top.vhd qmtech_demo_top.vhd
    struct_tb.vhd flow_tester_tb.vhd} {
    vcom -2008 [file join $source_dir $source]
}
set BreakOnAssertion 2
foreach cycles {1 2 3 8} {
    vsim -onfinish stop work.struct_tb -gREJECT_CYCLES=$cycles
    run -all
    quit -sim
}
foreach cycles {1 2 8} {
    vsim -onfinish stop work.flow_tester_tb -gSTEP_CYCLES=$cycles
    run -all
    quit -sim
}
```

Приложение 8. Файл qmtech_demo.qsf

Файл настроек взят из текущего проекта. Контакты и стандарт IO сверены с опубликованной схемой V03 и проверены полной компиляцией; перед прошивкой нужно подтвердить ревизию своей платы.

```
set_global_assignment -name FAMILY "Cyclone 10 LP"
set_global_assignment -name DEVICE 10CL080YU484C8G
set_global_assignment -name TOP_LEVEL_ENTITY qmtech_demo_top
set_global_assignment -name VHDL_INPUT_VERSION VHDL_2008
set_global_assignment -name VHDL_FILE conveyor_sorter.vhd
set_global_assignment -name VHDL_FILE flow_tester.vhd
set_global_assignment -name VHDL_FILE qmtech_demo_top.vhd
set_global_assignment -name SDC_FILE qmtech_demo.sdc
set_location_assignment PIN_G1 -to clk50_i
set_location_assignment PIN_P4 -to nRst_btn_i
set_location_assignment PIN_V10 -to conveyorLed_n_o
set_location_assignment PIN_U9 -to rejectLed_n_o
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to clk50_i
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to nRst_btn_i
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to
conveyorLed_n_o
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to rejectLed_n_o

set_instance_assignment -name PARTITION_HIERARCHY root_partition -to | -
section_id Top
set_global_assignment -name PARTITION_NETLIST_TYPE SOURCE -section_id Top
set_global_assignment -name PARTITION_FITTER_PRESERVATION_LEVEL
PLACEMENT_AND_ROUTING
    -section_id Top
set_global_assignment -name PARTITION_COLOR 16764057 -section_id Top
set_global_assignment -name LAST_QUARTUS_VERSION "25.1std.0 Standard
Edition"
set_global_assignment -name NUM_PARALLEL_PROCESSORS 4
# On-board LEDs have 1 kOhm series resistors (QMTECH V03 schematic).
set_instance_assignment -name CURRENT_STRENGTH_NEW "4MA" -to
conveyorLed_n_o
set_instance_assignment -name CURRENT_STRENGTH_NEW "4MA" -to rejectLed_n_o
```

Приложение 9. Файл qmtech_demo.qpf

```
QUARTUS_VERSION = "22.1std"
PROJECT_REVISION = "qmtech_demo"
```

Приложение 10. Файл qmtech_demo.sdc

```
create_clock -name clk50 -period 20.000 [get_ports {clk50_i}]
derive_clock_uncertainty
# External button asynchronously asserts the reset synchronizer.
# Release is synchronized by resetSync_r before reaching the datapath.
set_false_path -from [get_ports {nRst_btn_i}]
# LEDs have no external sampling clock or setup/hold requirement.
set_false_path -to [get_ports {conveyorLed_n_o rejectLed_n_o}]
```